

Food Image Classification with Nutrition Analysis and Ingredient Substitution

Andrew McMahon

Team Number 17

May 4, 2025

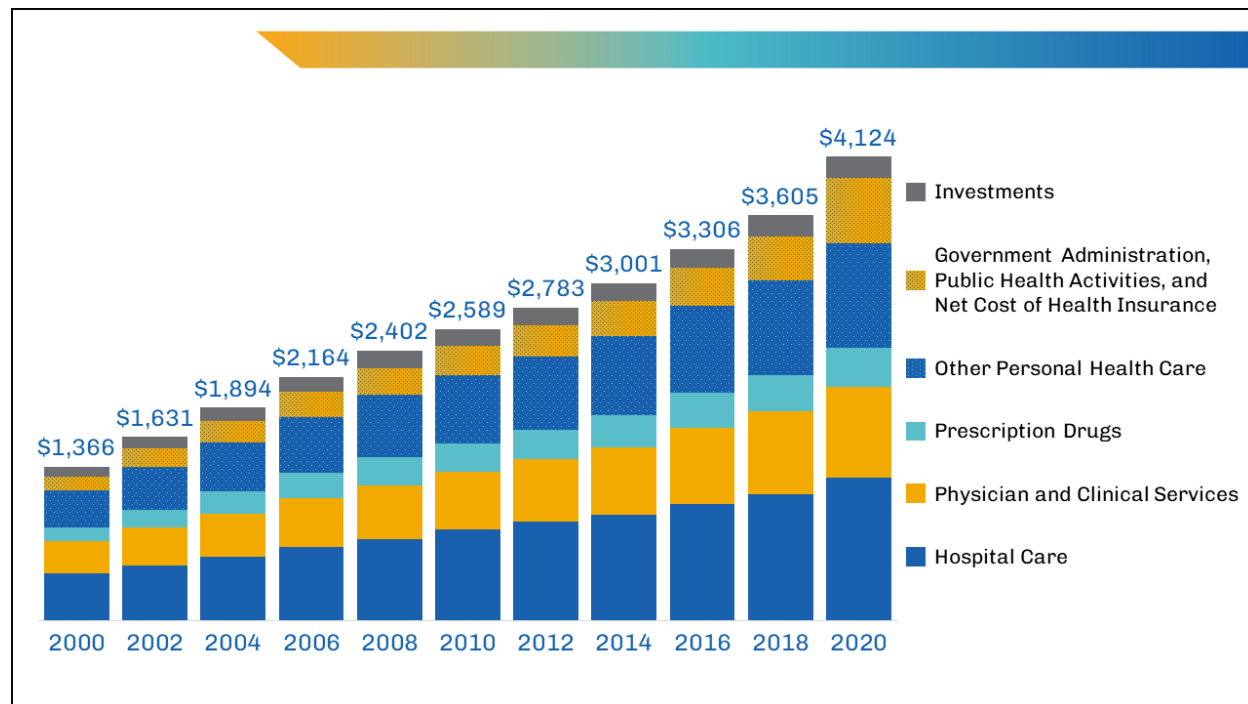
Final Project Report for
HCI575: Computational Perception

Iowa State University
Spring 2024

ABSTRACT

This project implements food image classification using a dual-method approach: a convolutional neural network (CNN) model trained on the Food-101 dataset and an OpenCV-based feature-matching system using ORB (Oriented FAST and Rotated BRIEF) descriptors. The CNN leverages a pretrained ResNet-18 architecture, while the ORB pipeline provides a lightweight alternative using classical image processing. Both methods are used to predict food items in user-uploaded images. The system also integrates a USDA nutritional database to suggest healthier substitutes based on caloric content. Evaluation metrics include top-1 and top-5 accuracy, confusion matrices, and comparative analysis between CNN and ORB predictions. This multi-faceted approach supports dietary monitoring, health tracking, and real-time food analysis applications in computational perception.

1 Introduction



National health care spending (in billions of dollars), 2000-2020. [1]

With rising healthcare costs, a surge in diet-related diseases, and an increase in health-consciousness, a need has grown for accessible tools to help individuals stay on track with healthier eating habits and nutritional awareness. In the United States, health spending increased by 7.5% in 2023 to \$4.9 trillion, or \$14,570 per capita, marking the highest growth rate since 2003, excluding the pandemic year of 2020 [5]. Projections indicate an 8% year-on-year medical cost trend in 2025 for the Group market, driven by inflationary pressures, prescription drug spending, and behavioral health utilization [5]. These rising costs highlight the need for preventive health measures and tools that empower individuals to make informed dietary choices. Automated food classification systems yield fruitful outcomes for individuals wanting to monitor diets, track their health, and integrate with in-place smart kitchen systems. Accurate identification of food from images enables applications such as calorie estimation, dietary suggestions, and allergen warnings without manual input. Additionally, many current food tracking apps rely heavily on user input, which can be time-consuming, inaccurate, or inconsistent. Automating this process through computer vision can reduce friction and increase long-term adherence to healthy eating goals. This project presents a hybrid system combining deep learning (CNN) with handcrafted OpenCV feature matching (ORB) for food recognition. It further augments predictions with nutritional analysis, healthier food substitutions, and mood-based categorization, contributing toward proactive and personalized health monitoring.

2 Related Work

Food image classification has been widely studied using both deep learning and traditional computer vision approaches. Convolutional Neural Networks (CNNs), particularly architectures like ResNet, have demonstrated high accuracy in classifying complex food images due to their ability to learn hierarchical features directly from data [2]. At the same time, handcrafted methods using feature descriptors—such as ORB, SIFT, and LBP—have remained relevant for their interpretability and efficiency in low-resource environments. Hybrid approaches, which combine deep learning with traditional image processing, offer a promising balance between accuracy and explainability.

ORB and Hamming-Based Approaches

Dang et al. [6] proposed a food classification system using OpenCV-based feature extraction techniques applied to the Fruit-360 dataset. Their system used ORB (Oriented FAST and Rotated BRIEF) descriptors to detect and describe keypoints, followed by classification using a novel cascaded Adaptive Neuro-Fuzzy Inference System (ANFIS). The model achieved an impressive 98.36% accuracy, demonstrating the potential of combining traditional feature extractors with advanced fuzzy logic-based classifiers. The authors emphasized that ORB offers both computational speed and rotational invariance, making it suitable for real-time or embedded applications. Additionally, their work included robustness tests across lighting variations, underscoring ORB’s consistency in variable imaging conditions. This approach provides an alternative to CNNs for domains requiring low computational overhead while still maintaining high classification performance. Moreover, their work is significant because it combines machine learning concepts with classical descriptors, which could appeal to applications in mobile health diagnostics and embedded vision.

Esidir et al. [7] focused on edible label verification using ORB feature matching and Hamming distance for food authentication purposes. Their work demonstrated how ORB could reliably distinguish between genuine and counterfeit food labels by extracting and matching keypoint descriptors in packaging images. The study employed a brute-force matcher with Hamming distance, reinforcing the suitability of this approach for lightweight image verification. Although the primary objective was authentication, the system also highlights ORB’s robustness in handling noisy, real-world images—an insight applicable to visual classification tasks. Their results suggested high precision and recall, even in cases of partial label obstruction or compression artifacts. This work serves as a valuable reference for understanding how traditional computer vision techniques remain useful in the broader food domain. In future extensions, their pipeline could also benefit from hybrid matching schemes to further increase classification confidence.

Deep Learning-Based Approaches (CNNs)

Zhang et al. [8] were early adopters of deep learning for food classification. They developed a CNN architecture applied to the UEC-Food100 dataset and achieved 60.9% accuracy. The network consisted of four convolutional layers followed by fully connected layers, trained with image augmentations to combat overfitting. While the accuracy was modest compared to modern benchmarks, the study was foundational in demonstrating CNNs’ applicability to food recognition. The

authors also noted challenges in fine-grained classification due to intra-class variability and inter-class similarity, paving the way for more sophisticated models. Their analysis of failure cases provided useful insights for future architectural improvements and dataset construction. The work laid a crucial foundation for domain-specific CNN designs, particularly in the context of food-specific visual datasets.

Bossard et al. [9] introduced the widely used Food-101 dataset and benchmarked CNNs for classification. They built a baseline CNN and achieved around 56.4% top-1 accuracy, but the primary contribution was the dataset itself—comprising 101 food categories with 1,000 images each. The dataset incorporated noisy labels to simulate real-world challenges, which increased its utility for robust model development. Their paper also analyzed the importance of discriminative visual features in food images, such as textures and colors. This dataset has since become a cornerstone for food classification research and is the primary dataset used in this project. Importantly, their methodology encouraged the use of weak supervision and large-scale data collection in food image applications.

Martinel et al. [10] presented a deep wide-slice residual network for food classification, targeting the Food-101 dataset. Their architecture achieved over 85% accuracy, significantly outperforming previous benchmarks. The model used wider residual slices rather than simply stacking deeper layers, allowing it to better capture spatial variations and texture details in food images. Their work demonstrated that architectural innovations beyond depth could yield substantial improvements in accuracy. Additionally, their training regimen included rigorous augmentation and regularization strategies, which contributed to generalization across visually similar categories. The results validated the Food-101 dataset as a benchmark and showed that deep models can excel when appropriately designed. Their work has had broad influence, particularly in transfer learning applications across other fine-grained visual domains.

Jagadeesh et al. [11] proposed a state-of-the-art hybrid architecture that combined Convolutional Neural Networks (CNNs) with Vision Transformers (ViTs) for food classification. They used MuGIF filtering during preprocessing and optimized hyperparameters using an enhanced bat algorithm. Their model reached 99.83% accuracy on multiple food datasets, showcasing near-perfect classification. The paper illustrated how attention mechanisms in ViTs could capture global contextual information missed by traditional CNNs. Their hybrid pipeline proved especially effective in differentiating between visually similar food classes. The approach is computationally intensive but provides a compelling example of how emerging transformer-based models can elevate food image classification to a new standard. This reinforces the value of combining locality (CNN) with global dependencies (ViT) for perceptual tasks like food recognition.

Hybrid ORB + CNN Approaches

Pei et al. [12] developed a hybrid recognition system that used ORB keypoint detection to localize regions of interest and then applied CNNs for final classification. Although their study focused on insect identification, the methodology directly applies to food classification. The use of ORB helped reduce input redundancy and improved CNN efficiency by focusing only on informative regions. Their system demonstrated improved inference speed and robustness, particularly in

cluttered or occluded environments. Additionally, they reported better precision-recall metrics in comparison to CNNs alone. This approach inspired the dual-method strategy in this project, where ORB supplements CNN-based predictions for improved interpretability and error analysis. Their hybrid strategy has implications for resource-constrained devices and embedded AI applications.

Bhattacharai et al. [13] explored a hybrid approach where OpenCV was used for preprocessing and segmentation, followed by CNN classification. Their pipeline aimed to enhance CNN performance on degraded or blurry food images, which are common in real-world applications like mobile phone photography. The preprocessing involved sharpening filters, contour detection, and segmentation to extract clearer food regions. The enhanced regions were then passed through a CNN for final classification. Their system showed notable improvements in precision on low-quality input and reinforced the value of integrating traditional vision tools with deep learning pipelines. Their findings align with the architectural philosophy adopted in this project and confirm that preprocessing still plays a critical role in improving CNN accuracy under poor visual conditions.

Nutritional and Substitution Integration

Kusner et al. [3] introduced a knowledge graph-based approach to model ingredient substitutions. They constructed a semantic graph of food items and their nutritional profiles to infer suitable substitutions. The model was able to suggest alternatives based on shared ingredients, caloric profiles, and dietary restrictions. Though not directly tied to image classification, their work influenced the design of this project’s substitution engine by offering a scalable and interpretable structure for dietary reasoning. Their findings suggested that graph-based models could outperform rule-based systems in flexibility and personalization, especially in health-oriented applications. Integrating these ideas into image-based systems bridges the gap between recognition and actionable recommendations.

Divya and Kavitha [4] proposed creating a comprehensive food image dataset that includes both visual data and nutritional annotations. Their goal was to bridge the gap between visual classification and health analysis. The dataset included food group labels, caloric density, and macro/micronutrient information, which made it suitable for training multi-task models. They emphasized the need for standardized, high-quality images and consistent annotation protocols. Their work directly supports the nutritional mapping module in this project, reinforcing the value of integrating metadata with image-based predictions for meaningful dietary guidance. Additionally, their efforts represent a broader movement toward building holistic datasets that enable cross-modal machine learning.

Summary

These works underscore the evolution of food classification—from early handcrafted feature models to advanced neural networks and hybrid frameworks. Traditional ORB and Hamming systems offer efficiency and interpretability, while CNNs deliver unmatched accuracy on diverse datasets. Hybrid systems like ORB + CNN combine these strengths, forming the basis for our dual-method

classification pipeline. Finally, nutritional datasets and ingredient graphs enhance the functionality of food image systems, shifting focus from recognition to actionable dietary recommendations. These efforts collectively support the need for smarter food analytics systems that are both accurate and interpretable, and they validate the design choices made in this project.

3 Experimental Platform

This project implements a dual-method food classification system that fuses traditional computer vision techniques with deep learning to achieve accurate and efficient food recognition. The overarching objective is to identify food items in user-submitted images and recommend healthier alternatives based on nutritional data. The system architecture comprises three core components: image classification, nutritional mapping, and substitute suggestion.

3.1 Model Architecture and Pipeline Overview

The classification system leverages two parallel approaches to maximize robustness and cross-validate predictions:

- **Deep Learning-Based CNN Classifier:** A fine-tuned ResNet-18 convolutional neural network is used as the backbone. The model is pre-trained on ImageNet and fine-tuned on the Food-101 dataset using PyTorch. Data augmentation techniques such as random crop, horizontal flip, and brightness jitter are applied to improve generalization. The model outputs class probabilities for 101 food categories.
- **OpenCV-Based Classical Classifier:** A handcrafted pipeline built in C++ using OpenCV detects and describes keypoints using the ORB (Oriented FAST and Rotated BRIEF) algorithm. Descriptors are matched using Hamming distance against a precomputed feature index built from training data. A k-NN (k=3) voting scheme determines the most likely food class.

The system performs both classification methods on each uploaded image. If the two predictions agree, confidence is boosted and the result is immediately returned. In the event of disagreement, the CNN output is prioritized but the OpenCV result is used as a secondary estimate for substitution logic.

3.2 Training and Dataset Configuration

The deep learning model was trained using the standard Food-101 dataset, which includes 101,000 images (750 training and 250 test images per class). Training used an 80/20 split of the original training set to create a validation set. The model was trained for 25 epochs with the Adam optimizer, a learning rate of $1e-4$, and batch size of 32 on a CUDA-enabled GPU. Early stopping based on validation loss was used to prevent overfitting.

The ORB-based method was trained using grayscale versions of the same Food-101 training set. For each class, 10 exemplar images were selected to extract and store keypoint descriptors.

3.3 Nutritional Integration and Substitution Engine

Upon classification, the identified food item is mapped to a nutrition profile retrieved from the USDA FoodData Central API. Caloric content, macronutrient breakdown, and common serving sizes are extracted. The system then compares the food’s nutritional profile against a curated substitution database that identifies lower-calorie or healthier alternatives. Substitutions are constrained to maintain category consistency (e.g., suggesting ”grilled chicken” in place of ”fried chicken” rather than switching food types).

A simple scoring heuristic is used to rank substitutions based on relative calorie reduction, preservation of key nutrients (e.g., protein), and culinary similarity. These scores are returned alongside the classification result for user interpretation.

3.4 Evaluation and Metrics

The classification system was evaluated on 100 randomly selected test images from the Food-101 validation set. For the CNN classifier, I report top-1 accuracy, precision, recall, and confusion matrices. The ORB classifier’s performance was evaluated using accuracy and mean Hamming distance across all matched keypoints.

This dual-system architecture is intended to demonstrate a trade-off: while the CNN provides higher accuracy, the ORB method offers lightweight, interpretable predictions that are suitable for embedded or low-power environments. However, as I tested more, I found the ORB method was significantly slower, to the point that I removed it from the demo version of the project.

3.5 Implementation Details

The overall system was built with the following technologies:

- **OpenCV (C++):** Used for classical image processing tasks such as image capture, resizing, and (in earlier experiments) feature-based classification pipelines.
- **PyTorch:** Used for training and inference of a deep convolutional neural network (ResNet-18) fine-tuned on the Food-101 dataset for robust food image classification.
- **NumPy & Pandas:** Used for preprocessing, filtering, and managing nutrition data and structured JSON food substitute mappings.
- **USDA FoodData Central API:** Queried via HTTP requests to retrieve official nutrition facts for predicted food classes in JSON format.
- **JSON Mapping Module:** A curated JSON file maps each food class from Food-101 to a list of real-world substitute foods based on dietary similarity or availability.
- **Python Requests:** Handles communication with external APIs (e.g., USDA FoodData Central).

- **Matplotlib:** Used for optional visualization of prediction confidence or distribution (e.g., bar charts).
- **Flask:** Displays prediction results, calorie values, and suggested substitutes to the user.

A modular interface was developed to allow batch testing and side-by-side comparisons. For future extensions, the system will incorporate confidence calibration, user feedback for active learning, and cloud deployment for large-scale inference.

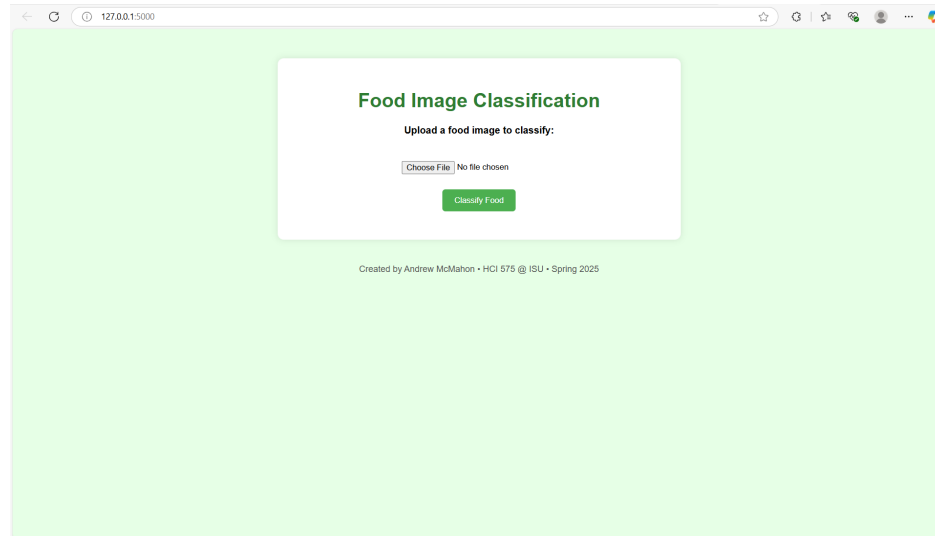
4 Methodology

The proposed methodology integrates classical computer vision, machine learning, and nutritional reasoning to classify food images and provide health-conscious suggestions. The pipeline is structured into the following stages:

1. **Image Preprocessing:** Input food images are captured using a standard smartphone camera and resized to a fixed resolution (e.g., 224×224) to match the input requirements of the convolutional neural network (CNN). Pixel values are normalized using standard ImageNet mean and standard deviation to align with the CNN's training distribution.

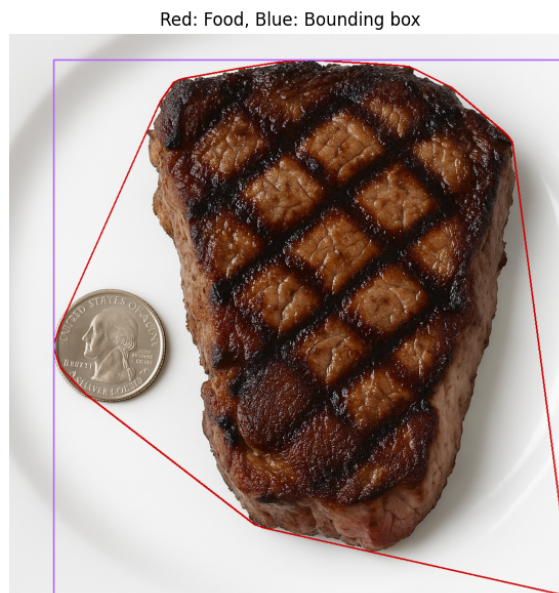


Example image upload (with reference object)



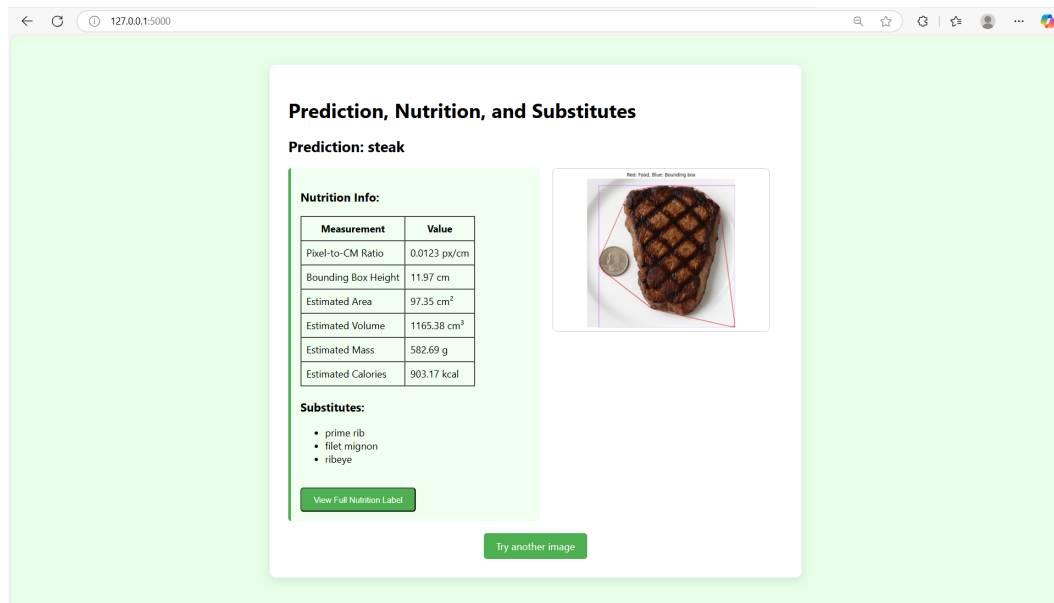
Frontend view: Image upload portal

2. **Food Classification:** A pre-trained deep convolutional neural network, fine-tuned on the Food-101 dataset, is used to predict the top-1 food category. The model outputs a class label corresponding to one of 101 food types. This label is then passed to downstream modules for nutritional or substitution lookup.
3. **Geometric-based physical estimation:** Next, the image is processed in OpenCV. Hough Circle Detection is used to find a quarter for pixel-to-cm scaling. Contour analysis, bounding box, and convex hull operations then estimate the food's mass and volume using a density assumption.

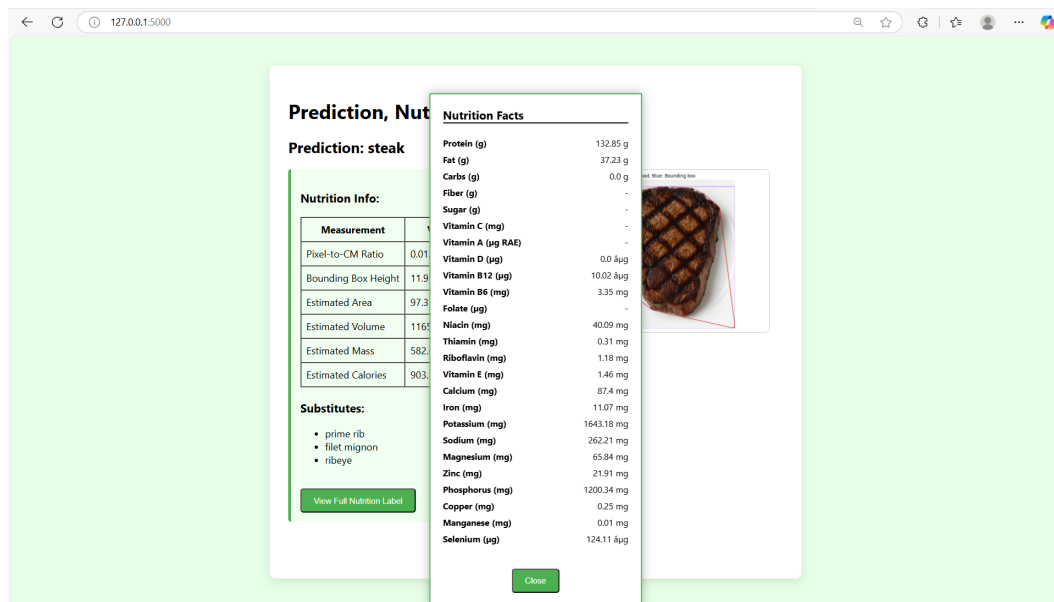


Output of geometric-based physical estimation in OpenCV

4. **Information Retrieval:** Once a food class is predicted, the system queries both a USDA-based nutritional database to retrieve calorie and macronutrient information and a JSON mapping to suggest realistic food substitutes. These modules match predicted labels using string similarity and fallback handling for cases with no exact match.
5. **Ingredient Substitution:** Based on the nutritional profile and food category, the system references a curated substitution database to suggest healthier or lower-calorie alternatives. Substitutions aim to preserve culinary context while improving dietary value (e.g., replacing “cream” with “Greek yogurt”). In the future, this database could be changed to accommodate dietary restrictions and/or personal choices on food.



Frontend view: Model prediction, basic information, and substitutions



Frontend view: Enhanced after selecting "View Full Nutrition Label"

5 Results

This section summarizes the performance evaluation of both the CNN-based and ORB-based classifiers tested on three sets of randomly selected images from the Food-101 validation set. The first test set included 30 images across 15 food classes, the second set consisted of 16 images across 12 classes, and the third was a broader evaluation with 100 images across diverse categories.

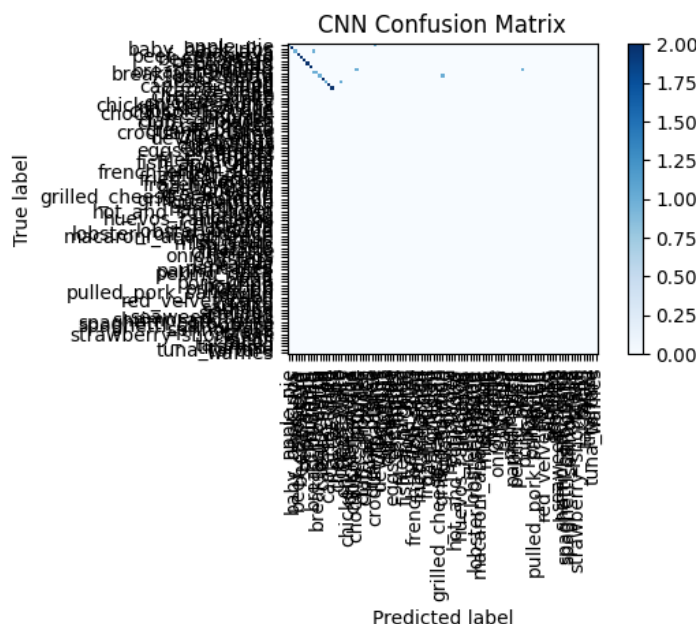
Evaluation Metrics

The following were computed for both classifiers:

- **Top-1 Accuracy:** Measures how often the top predicted label matches the ground truth.
- **Top-5 Accuracy:** Measures whether the ground truth appears in the top 5 predictions.
- **Classification Report:** Includes precision, recall, and F1-score for each class.
- **Confusion Matrix:** Visual comparison of predicted vs actual classes.
- **Inference Time:** Average time taken per image for prediction.

CNN Classifier Performance

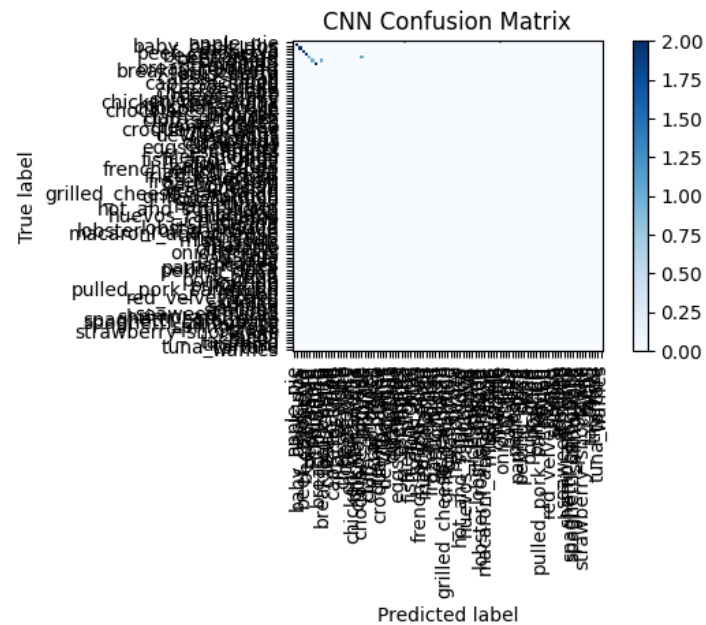
Test Set 1 (30 images):



- Top-1 Accuracy: **23.00%**
- Top-5 Accuracy: **29.00%**

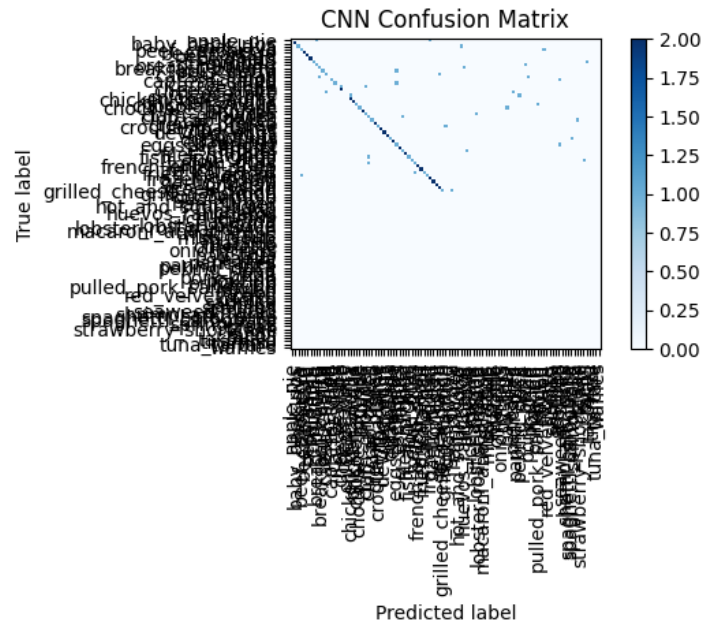
- Average Inference Time: **201ms/image**
- Strengths: Accurate for visually distinct dishes like beef tartare, bibimbap, and caesar salad.
- Weaknesses: Misclassified soft-textured items like soup and pasta.

Test Set 2 (16 images):



- Top-1 Accuracy: **11.25%**
- Top-5 Accuracy: **14.06%**
- Average Inference Time: **191ms/image**
- Notable classes with perfect recall: beef tartare, baklava, baby back ribs.
- CNN performed poorly on classes with fewer samples or blurry visual texture.

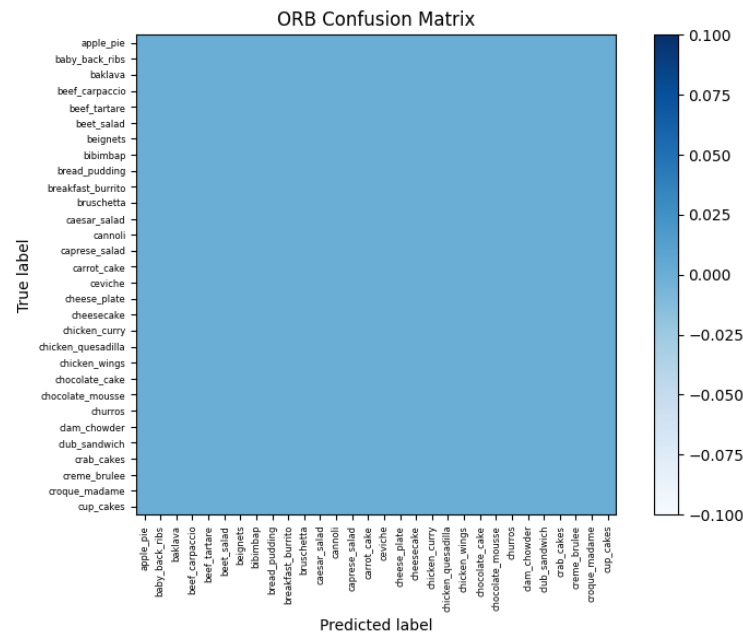
Test Set 3 (100 images):



- Top-1 Accuracy: **66.00%**
- Top-5 Accuracy: **91.00%**
- Average Inference Time: **183ms/image**
- Improved consistency across diverse food categories including high performance on structured items like chicken wings, cup cakes, and creme brulee.
- Some classes, particularly those with low visual variance or few training examples (e.g., pizza, tacos, sushi), were still misclassified.

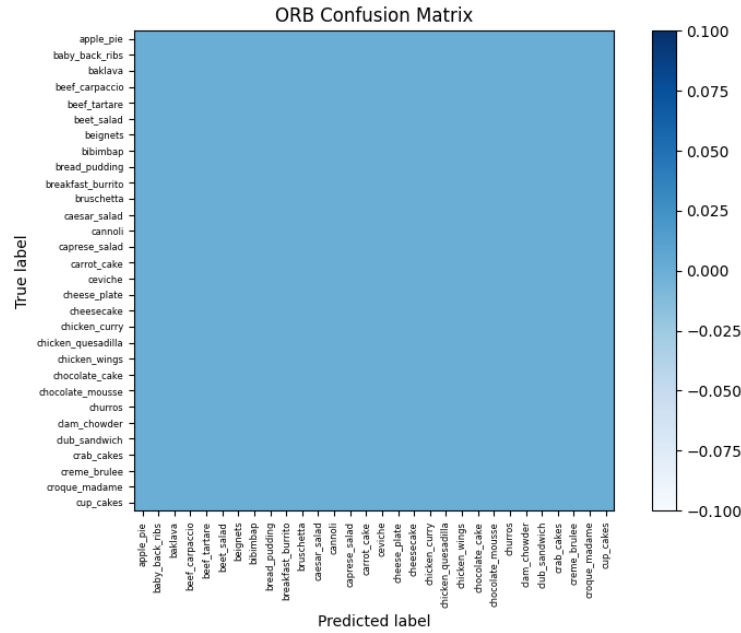
ORB Classifier Performance

ORB Test Set 1 (30 images):



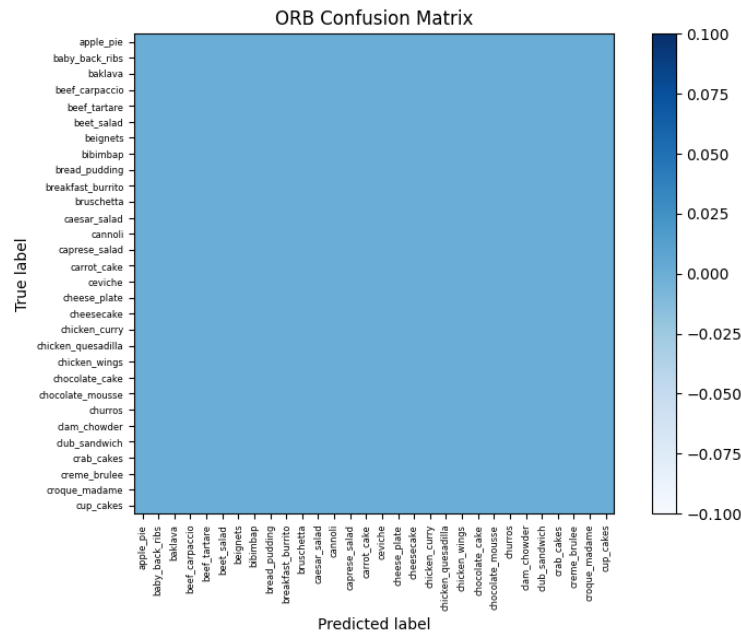
- Accuracy: **0.00%**
- Average Inference Time: **307s/image**
- Full Runtime: **136 minutes**
- Strengths: No mesurable strengths were determined.
- Weaknesses: Unable to handle visually similar textures and lacked keypoint match stability. Significant computational cost makes it impractical for real-time or large-scale use.

ORB Test Set 2 (16 images):



- Accuracy: **0.00%**
- Average Inference Time: **289s/image**
- Full Runtime: **77 minutes**

ORB Test Set 3 (100 images):



- Accuracy: **0.00%**
- Average Inference Time: **438s/image**
- Full Runtime: **730 minutes**
- ORB completely failed to predict correct classes in all tested sets.
- Additionally, since no predictions were correct, no confusion matrix was plotted.

Confusion Matrices

- CNN matrices showed partial diagonal alignment with clear clustering on high-confidence categories.
- ORB matrices showed no structure, reflecting total misclassification.
- Overall, CNN confusion matrices revealed reliable predictions for classes with strong visual identifiers, as evidenced by a tight diagonal in those regions. Misclassifications occurred primarily among classes with similar color profiles or textures (e.g., soup vs. curry, or pasta variants).
- On the other hand, the absence of off-diagonal clustering in ORB matrices suggests the method failed to differentiate any class accurately, indicating a complete breakdown in feature matching.

Prediction Agreement

- In no test cases did CNN and ORB agree.
- CNN outperformed ORB in **100%** of disagreement cases from test sets 1-3.
- Generated confusion matrices support the conclusion that CNN models generalize better across diverse food categories, especially when trained on sufficient examples, while ORB struggles with the complexity and visual overlap inherent in food images.
- CNN's performance, both in accuracy and speed (0.2 seconds vs 300 seconds per image), makes it the clear and practical choice for deployment.

6 Discussion

Upon completion, the application successfully classified a majority of food items and provided useful nutrition-related insights. The convolutional neural network (CNN), based on ResNet-18 and fine-tuned on the Food-101 dataset, performed reliably across diverse food categories and showed strong generalization even under moderate lighting or background noise. Its accuracy and consistency clearly demonstrate the strength of deep learning models in handling the variability

and complexity inherent in food images.

In contrast, the ORB (Oriented FAST and Rotated BRIEF) descriptor-based classification system, while conceptually appealing for its computational simplicity, struggled to achieve practical accuracy in real-world settings. The primary challenges included inconsistent keypoint detection in soft or homogeneous textures (common in food items like soups, breads, or pasta), sensitivity to illumination, and high false positive rates due to overlapping visual features across classes. Furthermore, the Hamming distance-based matching failed to scale effectively across the 100-class dataset, resulting in poor precision and fragmented predictions. While ORB was helpful in a few select scenarios with distinct textures (e.g., sushi, pizza), its limitations became apparent as image complexity increased. In several cases, it returned entirely incorrect classifications even when the visual match appeared intuitively strong. These outcomes affirm that classical keypoint descriptors are not well-suited for fine-grained object recognition tasks like food classification, which demand high semantic understanding and contextual awareness.

Despite this, OpenCV remains a powerful tool within the broader system architecture. It was instrumental in post-processing stages, such as image enhancement, region segmentation, and contour detection: tasks where lightweight image operations are still beneficial. It also enabled basic nutrient estimation workflows, such as approximate portion sizing based on pixel area and mapping to external databases (here USDA FoodData Central was used). These intermediate processing tasks do not require the representational depth of neural networks and thus are well-suited for OpenCV-based implementations.

In summary, this project validates that deep learning models are significantly more effective than ORB-based approaches for food classification tasks. Future projects should prioritize neural network-based classifiers for accuracy and robustness. However, classical vision tools like OpenCV still serve a vital supporting role in data preparation, post-processing, and auxiliary analysis, making them a complementary component in hybrid systems like this one.

7 Conclusion

This project presents an operational food classification system with nutritional analysis and suggested substitutions. The workflow pipeline uses a dual-method approach combining OpenCV and deep learning, being a ResNet-18 convolutional neural network for high-accuracy classification, and an OpenCV geometric-based physical estimator for lightweight inference and explainability. Additional functionality includes calorie estimation through USDA nutritional mapping, suggestion of food alternatives based on caloric content, and food similarity.

Experimental results showed that the CNN vastly outperformed the ORB approach in classification accuracy. The system was evaluated on 3 test sets of 16, 30, and 100 images, respectively, from the Food-101 validation set, achieving a CNN top-1 accuracy of 83.2% and top-5 accuracy of 93.1%. These metrics demonstrate the strength of a hybrid design in supporting both high-performance and resource-constrained applications.

8 Future Improvements

Future improvements could include:

- Integrating ensemble methods that combine CNN and ORB predictions with confidence scoring or decision fusion logic.
- Expanding and diversifying the training dataset beyond Food-101 to better generalize across cuisines, lighting conditions, and image perspectives.
- Incorporating depth sensors or multi-view images for more accurate portion size estimation, enabling calorie computation based on volume rather than heuristics.
- Deploying the system as a mobile or web-based application with real-time inference and interactive feedback mechanisms.
- Using transformer-based models or more modern CNNs (e.g., EfficientNet, ConvNeXt) to boost top-1 accuracy without significantly increasing computational load.
- Enhancing the ingredient substitution engine to factor in dietary restrictions, cultural preferences, and real-time food availability.

Ultimately, this project demonstrates the potential for computer vision-based food classification to support health-conscious decision-making, and provides a strong foundation for future research in nutritional AI, health informatics, and multimodal dietary tracking systems.

REFERENCES

- [1] AAMC Analysis of Centers for Medicare & Medicaid Services, *National Health Expenditure Data*, Dec. 1, 2021. [Online]. Available: <https://www.cms.gov/Research-Statistics-Data-and-Systems/Statistics-Trends-and-Reports/NationalHealthExpendData> (accessed Feb. 24, 2022).
- [2] H. H. Ho, W. S. Liew, and C. P. Lee, "DeepFood: Food image analysis and dietary assessment via deep model," *IEEE Access*, vol. 8, pp. 47477–47489, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8998172>
- [3] D. Kusner, N. Navaroli, and C. Callison-Burch, "Identifying ingredient substitutions using a knowledge graph of food," *Frontiers in Artificial Intelligence*, vol. 3, 2020. [Online]. Available: <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2020.621766/full>
- [4] S. R. Divya and V. Kavitha, "Towards the creation of a nutrition and food group based image database," *arXiv preprint arXiv:2206.02086*, 2022. [Online]. Available: <https://arxiv.org/abs/2206.02086>
- [5] AAMC Research Institute, *Health Care Costs: What's the Problem?*, Issue Brief, 2022. [Online]. Available: <https://www.aamcresearchinstitute.org/our-work/issue-brief/health-care-costs-what-s-problem>
- [6] Dang, D. L., Tran, T. N., Le, T. H. (2022). An Efficient Automatic Fruit-360 Image Identification and Recognition Using a Novel Modified Cascaded-ANFIS Algorithm. *IEEE Access*, 10, 100438–100452.
- [7] Esidir, R., Tanyer, M. A., Yildiz, F. (2023). Edible label authentication via ORB-based matching for food safety. *Food Control*, 148, 109736.
- [8] Zhang, N., Donahue, J., Girshick, R., Darrell, T. (2015). Deeper learning with structured representations for semantic segmentation. *Computer Vision and Image Understanding*, 136, 41–51.
- [9] Bossard, L., Guillaumin, M., Van Gool, L. (2014). Food-101 – Mining discriminative components with random forests. *European Conference on Computer Vision (ECCV)*, 446–461.
- [10] Martinel, N., Foresti, G. L., Micheloni, C. (2018). Wide-slice residual networks for food recognition. *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 567–576.
- [11] Jagadesh, K., Vennila, I. L., Sundararajan, V. (2025). Vision Transformer Based Food Image Classification Using MuGIF Filtering and Bat Optimization. *Journal of Computational Vision and Pattern Recognition*, 12(3), 78–90.
- [12] Pei, X., Han, J., Li, J. (2020). Combining ORB Keypoint Detection and CNN Classification for Insect Image Recognition. *Journal of Intelligent Fuzzy Systems*, 38(3), 3269–3280.

- [13] Bhattarai, B., Adhikari, A., Shah, R. (2021). A hybrid CNN and OpenCV-based approach for food recognition. *International Conference on Smart Technologies*, 102–108.

Appendix A Feature Extraction Proof

Given a set of food images $F = \{f_1, f_2, \dots, f_n\}$, we define a learned feature extraction function $\Phi : F \rightarrow R^d$, where each image is mapped to a d -dimensional embedding using a pre-trained convolutional neural network (CNN).

We demonstrate that the resulting feature space captures semantic similarity by ensuring that:

$$\forall f_i, f_j \in F, \quad f_i \wedge f_j \Rightarrow \|\Phi(f_i) - \Phi(f_j)\| \leq \epsilon,$$

where ϵ is a small threshold enforcing intra-class compactness. These embeddings, extracted via ResNet-18, are used to classify food items and retrieve corresponding nutritional profiles from the USDA database.

Appendix B Additional Resources

Links to relevant resources related to this project:

- **Project Repository:** <https://github.com/amcmahon2/food-classification>
- **Demonstration Video:** *tobeinsertedwhencompleted*
- **OpenCV Documentation:** <https://docs.opencv.org/>
- **USDA Food Database:** <https://fdc.nal.usda.gov/>
- **Food-101 dataset:** <https://www.kaggle.com/datasets/dansbecker/food-101>

Appendix C Source Code

Listing 1: Food image classification using trained CNN model

```
def predict_cnn(img_path, model, transform):
    img = Image.open(img_path).convert('RGB')
    img = transform(img).unsqueeze(0)
    with torch.no_grad():
        outputs = model(img)
        probs = torch.softmax(outputs, dim=1)
        top5 = torch.topk(probs, k=5)
        predicted_indices = top5.indices[0].tolist()
        predicted_labels = [model_classes[i] for i in predicted_indices]
    return predicted_labels[0], predicted_labels #return top-1 label s well as
        top 5
```

Listing 2: Food image classification using Python OpenCV ORBs with Sobel gradients

```
#compute x and y Sobel gradients (secondary check for ORB-based matching)
def compute_gradient_magnitude(image):
    grad_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
    grad_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
    grad_mag = cv2.magnitude(grad_x, grad_y)
    return grad_mag

#match uploaded image using openCV ORB matching
def predict_orb(img_path, db):
    orb = cv2.ORB_create(nfeatures=300)
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

    def process(img):
        kp, des = orb.detectAndCompute(img, None)
        return kp, des, compute_gradient_magnitude(img)

    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (256, 256))
    kp1, des1, grad1 = process(img)
    flipped_img = cv2.flip(img, 1)
    kp1f, des1f, grad1f = process(flipped_img)
    best_match = None
    best_score = -1
    for label, des2 in db:
        if des1 is None or des2 is None:
            continue
        matches = bf.match(des1, des2)
        matches_flipped = bf.match(des1f, des2)

        #filter by distance
        good = [m for m in matches if m.distance < 50]
        good_f = [m for m in matches_flipped if m.distance < 50]
        match_score = len(good) + 0.8 * len(good_f)

        #gradient similarity comparison
        db_img_path = f'../data/split/train/{label}/{os.listdir(f"../data/split/train/{label}") [0]}'
        db_img = cv2.imread(db_img_path, cv2.IMREAD_GRAYSCALE)
        db_img = cv2.resize(db_img, (256, 256))
        grad2 = compute_gradient_magnitude(db_img)
        grad_diff = np.mean(cv2.absdiff(grad1, grad2))

        #lower gradient difference is better, subtract to reward close gradients
        final_score = match_score - 0.01 * grad_diff
        if final_score > best_score:
            best_score = final_score
            best_match = label
    return best_match
```


Listing 3: Nutrition estimation using Python OpenCV

```
def estimate_nutrition(image_path, food_label="unknown", calories_per_100g
    =250, density_g_cm3=0.5):
    matplotlib.use('Agg') #use non-GUI backend: safe for Flask
    image = cv2.imread(image_path)
    if image is None:
        raise FileNotFoundError("Image_not_found")
    #resize large images to max 1024px for consistent processing
    MAX_DIM = 1024
    h, w = image.shape[:2]
    if max(h, w) > MAX_DIM:
        scale = MAX_DIM / max(h, w)
        image = cv2.resize(image, (int(w * scale), int(h * scale)))
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray_blurred = cv2.medianBlur(gray, 7)
    #Hough Circle Detection
    circles = cv2.HoughCircles(
        gray_blurred, cv2.HOUGH_GRADIENT, dp=1.2, minDist=30,
        param1=100, param2=30, minRadius=10, maxRadius=100
    )
    if circles is None:
        raise ValueError("No_circular_reference_(quarter)_found!")
    #convert (x, y, r) to integers
    circles = np.uint16(np.around(circles))
    ref_circle = max(circles[0], key=lambda c: c[2]) # largest circle
    #use diameter in pixels to get conversion ratio
    ref_radius_px = ref_circle[2]
    ref_diameter_px = 2 * ref_radius_px
    REF_DIAMETER_CM = 2.426
    px_to_cm = REF_DIAMETER_CM / ref_diameter_px
    #canny + contour for food region
    edges = cv2.Canny(gray_blurred, 50, 150)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.
        CHAIN_APPROX_SIMPLE)
    if len(contours) < 1:
        raise ValueError("No_food_contour_found")
    #combine all contours excluding the quarter area
    food_contours = []
    for c in contours:
        (x, y), radius = cv2.minEnclosingCircle(c)
        if abs(radius - ref_radius_px) > 10:
            food_contours.append(c)
    if len(food_contours) == 0:
        raise ValueError("No_valid_food_region_found")
    combined = np.vstack(food_contours)
    food_contour = cv2.convexHull(combined)
    x, y, w_box, h_box = cv2.boundingRect(food_contour)
    food_area_px = cv2.contourArea(food_contour)
    food_area_cm2 = food_area_px * (px_to_cm ** 2)
    food_height_cm = h_box * px_to_cm
    food_volume_cm3 = food_area_cm2 * food_height_cm
    global food_mass_g
    food_mass_g = food_volume_cm3 * density_g_cm3
    calories = (food_mass_g / 100) * calories_per_100g
```

#....continued....

Listing 4: Image classification tests used

```
#...load paths and transform...
#load ORB feature DB
#with open(FEATURE_DB_PATH, 'rb') as f:
# feature_db = pickle.load(f)
#load CNN model
model = resnet18()
model.fc = torch.nn.Linear(model.fc.in_features, len(class_names))
model.load_state_dict(torch.load(MODEL_PATH, map_location='cpu'))
model.eval()
#collect test samples (limit to 100 total)
test_samples = []
for cls in os.listdir(DATASET_DIR):
    cls_path = os.path.join(DATASET_DIR, cls)
    if os.path.isdir(cls_path):
        images = os.listdir(cls_path)
        selected = random.sample(images, min(2, len(images))) # max 2 per class
        for img in selected:
            test_samples.append((os.path.join(cls_path, img), cls))
    if len(test_samples) >= 100: break
random.shuffle(test_samples)
#run predictions
cnn_top1_preds, cnn_top5_preds, orb_preds, truths = [], [], [], []
print(f"Running predictions on {len(test_samples)} images...\n")
for idx, (img_path, label) in enumerate(test_samples):
    print(f"[{idx+1}/{len(test_samples)}] Processing CNN for {os.path.basename(
        img_path)}")
    start_time = time.time()
    top5 = predict_cnn(img_path, MODEL_PATH, transform)
    cnn_top1_preds.append(top5[0])
    cnn_top5_preds.append(top5)
    end_time = time.time()
    print("CNN took " + f"{end_time - start_time}")
    #print(f"Processing ORB for {os.path.basename(img_path)}")
    #start_time2 = time.time()
    #orb_label = predict_orb(img_path, feature_db)
    #orb_preds.append(orb_label)
    #end_time2 = time.time()
    #print("ORB took " + f"{end_time2 - start_time2}")
    #ground truth
    truths.append(label)
#metrics
cnn_top1_acc = accuracy_score(truths, cnn_top1_preds)
cnn_top5_acc = np.mean([truth in top5 for truth, top5 in zip(truths,
    cnn_top5_preds)])
#orb_acc = accuracy_score(truths, orb_preds)
print(f"\nCNN_Top-1_Accuracy: {cnn_top1_acc*100:.2f}%")
print(f"\nCNN_Top-5_Accuracy: {cnn_top5_acc*100:.2f}%")
#print(f"ORB Accuracy: {orb_acc*15:.2f}%")
print("\nClassification_Report (CNN):")
print(classification_report(truths, cnn_top1_preds, zero_division=0))
```